Chit Chat — a Distributed Chat Service in Go

Course assignment: distributed systems — communication, coordination and ordering of events
Language: Go 1.27 · RPC: gRPC + Protocol Buffers · Timestamps: Lamport logical clocks
Repository: `~/Projects/chit-chat` (link to be inserted on LearnIT hand-in)

1. Streaming model — which direction?

We use **server-side streaming** for the `Join` RPC, and **unary** request/reply for `Publish` and `Leave`. We deliberately do **not** use client-side or bidirectional streaming, for three reasons:

- **Asymmetry of traffic.** Every client only ever *receives* a continuous flow of broadcasts; the only things a client sends are discrete, infrequent events (a published message, a leave). A stream in the client→service direction would carry one message and close, i.e. it is a unary call in all but name.
- **Server-side streaming is the natural fit for fan-out.** The service is the single origin of all broadcasts (joins, leaves, chat). Opening one long-lived server→client stream per participant lets the service push every event to every participant with a single, stable transport — no per-event RPCs to establish, no polling.
- **Bidirectional streaming buys nothing here.** gRPC bidirectional streaming is useful when both directions carry a continuous flow (e.g. live video, or an order book). Chit Chat has no such client-side flow, so a single stream on the receive side plus unary calls on the send side is the minimal design that satisfies S3–S6.

2. System architecture

The system is a **client–server** architecture (not peer-to-peer):

- **One service process** (the *Chit Chat service*) owns the participant registry, the Lamport clock, and the broadcast fan-out. It listens on a TCP port and serves all gRPC traffic. Clients never talk to each other — every message is relayed by the service.
- **Many client processes** (one per participant) join, publish and leave via gRPC. Each is an independent OS process, satisfying S2 (≥ 3 concurrent clients in the demo).
- **Why not P2P?** A pure peer-to-peer design would require every client to maintain N−1 connections, implement its own fan-out and gossip its clock, which multiplies the ordering problem. Centralising the clock and the fan-out in one service process gives a single, well-defined point where Lamport timestamps are assigned, which is exactly what makes the logical-time ordering of S4–S6 provable. The cost — the service being a single point of failure — is accepted in this assignment's scope (S1–S7 do not require fault tolerance of the service itself).

Concurrency inside each process uses **goroutines and channels** as required: the service spawns one dedicated goroutine per client connection (its `Join` handler plus that connection's reader goroutine), and each participant owns a private **buffered channel** into

which broadcasts are written with **non-blocking sends**. A slow or dead client therefore can never block delivery to the others, satisfying the "multiple concurrent clients without blocking message delivery" requirement.

3. RPC methods and message types

All definitions live in `grpc/chit_chat.proto`; the generated Go bindings are committed under `grpc/`. The service exposes exactly three methods:

Method	Type	Request	Response	Purpose
`Join`	**server-side streaming**	`JoinRequest` { name }	`stream BroadcastMessage`	Register the participant; the stream then carries *every* subsequent broadcast (joins, leaves, chat), including the join event itself (S5).
`Publish`	unary	`PublishRequest` { name, message ≤ 128 chars, lamport }	`PublishResponse` { logical_time }	Publish one valid UTF-8 chat message; the service stamps and broadcasts it (S3, S4).
`Leave`	unary	`LeaveRequest` { name, lamport }	`LeaveResponse` { logical_time }	Orderly departure; the service broadcasts the leave event to all remaining participants (S6).

Message types. `BroadcastMessage` is the single type pushed on the Join stream and carries: `sender` (the participant the event is about), `content` (human-readable text, e.g. "*Participant X joined Chit Chat at logical time L*" or "*Alice: hello*"), `logical_time` (the Lamport timestamp), and `kind` (an enum `CHAT | JOIN | LEAVE`) so clients can render categories distinctly. `JoinRequest`, `PublishRequest`, `PublishResponse`, `LeaveRequest` and `LeaveResponse` are the unary envelopes; the two request types carry the sender's current Lamport clock so the service can compute causally consistent timestamps (section 4).

4. Lamport timestamp calculation

Timestamps are **Lamport logical timestamps**: every process keeps a counter, and for each event (locally generated or received) it updates `T = max(local, carried) + 1`. In Chit Chat the **service assigns every timestamp**, which is what makes the ordering total and verifiable:

- **Service side (server/main.go, `hub.tick`).** The service holds the single authoritative clock. On every join, publish and leave it computes `T = max(serviceClock, carried) + 1`, where `carried` is the client's clock value sent in `PublishRequest.lamport` /

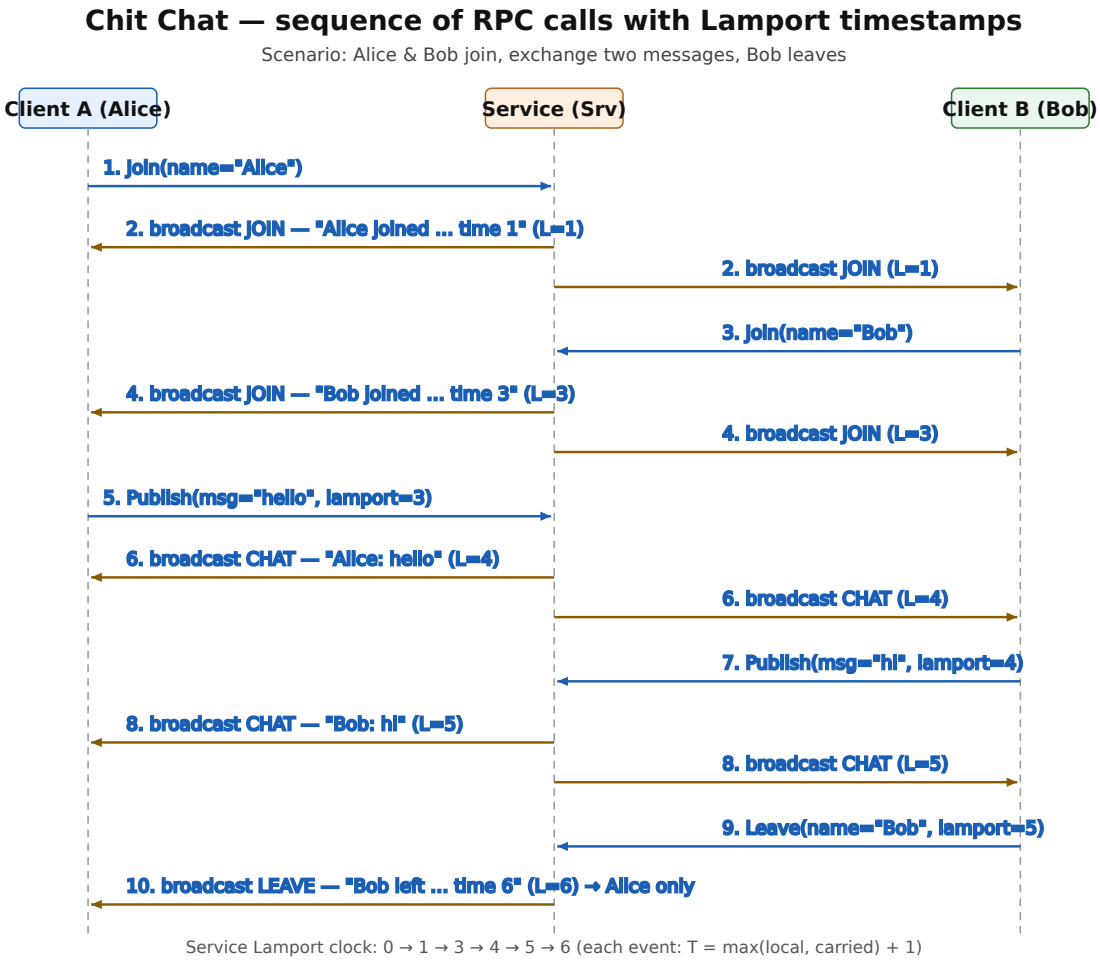
`LeaveRequest.lamport`. The clock only ever increases, so logical times are strictly ordered.

- **Client side (client/main.go).** Each client keeps its own Lamport clock and advances it on every broadcast received and every message sent (`clock = max(clock, carried)`). It reports this clock in every outgoing RPC. Because clients may publish in quick succession (their local ordering is unobservable to the service), carrying the client's clock in lets the service fold the client's local ordering into the global one, preserving causality across processes even though no shared physical clock is used.

> **Worked example from the demo run.** The service clock walks `0 → 1 → 3 → 4 → 5 → 6` as: Alice joins (t=1), Alice publishes (t=2), Bob joins carrying his clock (t=3), Alice publishes (t=4), Bob publishes carrying 4 (t=5), Bob leaves (t=6). Each value is `max(local, carried)+1`; the full capture is in the appendix and in `logs/`.

5. Sequence of RPC calls with Lamport timestamps

The figure below traces one concrete scenario — **Alice and Bob join, exchange two messages, Bob leaves** — showing every RPC, the direction of each broadcast, and the logical time assigned to each event. Arrows from the service (orange) are the broadcast fan-outs; blue arrows are the client→service RPCs.



Every broadcast is delivered to **all active participants**, including the actor of the event (Alice receives her own join at t=1; Bob receives his own join at t=3), and the leave at t=6

goes only to the remaining participant (Alice) — exactly S5 and S6.

6. Logging

All logging uses the standard `log` package, with a small structured wrapper (`logger.Event`) that emits one line per event in the form:

```
2026-10-25 17:21:07 component=Server event=broadcast_join client=Alice time=1 delivered=1
```

Every line carries the four required fields — **timestamp** (from `log.LstdFlags`), **component** (`Server` / `Client`), **event type** and **identifiers** (participant name, logical time, delivery count). The events logged are: server startup/shutdown, client connection and disconnection, join/leave broadcast, chat broadcast, and per-recipient delivery (`message_delivered` on the server, `message_received` on the client — together proving S7, which requires both display *and* logging of every broadcast).

7. Requirements traceability

Req	Where it is met	Evidence in `logs/`
S1	gRPC for all communication; API in `grpc/chit_chat.proto`	every event is an RPC
S2	1 service + 3 client processes in `test/run_demo.sh`	`client_connected client=Alice/Bob/Carol`
S3	`Publish` validates UTF-8 and ≤ 128 chars (server side)	`broadcast_chat … chars=N`
S4	every `broadcast_*` lists `delivered=N` = active clients, each with a `time=L`	all `broadcast_*` lines
S5	`Join` registers first, then broadcasts join to all incl. self	`broadcast_join … delivered=1` (Alice)
S6	`Leave` broadcasts to all remaining	`broadcast_leave … delivered=2` (Bob)
S7	client displays `[t=N] …` and logs `message_received`	each `client-*.log`
Logging	startup/shutdown, connect/disconnect, broadcasts, delivery — with ts, component, event, id	full `server.log`

8. How to run

```
# terminal 1 — the service
go run ./server                       # or: CHITCHAT_PORT=6000 go run ./server

# terminals 2-4 — three concurrent clients
go run ./client Alice
```

```
go run ./client Bob localhost:50051
go run ./client Carol

# or the whole scripted 3-client scenario in one go:
bash test/run_demo.sh                 # writes the evidence to logs/
```

Appendix A — captured logs (evidence for S5–S7 and the logging spec)

Produced by `bash test/run_demo.sh` (server + 3 concurrent clients; Alice, Bob, Carol).

A.1 Server log — `logs/server.log`

```
2026/10/25 17:21:07 component=Server event=server_startup port=50051
2026/10/25 17:21:07 component=Server event=client_connected client=Alice
2026/10/25 17:21:07 component=Server event=broadcast_join client=Alice time=1 delivered=1
2026/10/25 17:21:07 component=Server event=broadcast_chat client=Alice time=2 chars=14
delivered=1
2026/10/25 17:21:07 component=Server event=message_delivered client=Alice kind=JOIN time=1
2026/10/25 17:21:07 component=Server event=message_delivered client=Alice kind=CHAT time=2
2026/10/25 17:21:08 component=Server event=client_connected client=Bob
2026/10/25 17:21:08 component=Server event=broadcast_join client=Bob time=3 delivered=2
2026/10/25 17:21:08 component=Server event=message_delivered client=Alice kind=JOIN time=3
2026/10/25 17:21:08 component=Server event=message_delivered client=Bob kind=JOIN time=3
2026/10/25 17:21:08 component=Server event=broadcast_chat client=Bob time=4 chars=18 delivered=2
2026/10/25 17:21:08 component=Server event=message_delivered client=Alice kind=CHAT time=4
2026/10/25 17:21:08 component=Server event=message_delivered client=Bob kind=CHAT time=4
2026/10/25 17:21:08 component=Server event=broadcast_chat client=Alice time=5 chars=13
delivered=2
2026/10/25 17:21:08 component=Server event=message_delivered client=Alice kind=CHAT time=5
2026/10/25 17:21:08 component=Server event=message_delivered client=Bob kind=CHAT time=5
2026/10/25 17:21:08 component=Server event=client_connected client=Carol
2026/10/25 17:21:08 component=Server event=broadcast_join client=Carol time=6 delivered=3
2026/10/25 17:21:08 component=Server event=message_delivered client=Bob kind=JOIN time=6
2026/10/25 17:21:08 component=Server event=message_delivered client=Alice kind=JOIN time=6
2026/10/25 17:21:08 component=Server event=message_delivered client=Carol kind=JOIN time=6
2026/10/25 17:21:08 component=Server event=broadcast_chat client=Carol time=7 chars=4 delivered=3
2026/10/25 17:21:08 component=Server event=message_delivered client=Alice kind=CHAT time=7
2026/10/25 17:21:08 component=Server event=message_delivered client=Carol kind=CHAT time=7
2026/10/25 17:21:08 component=Server event=message_delivered client=Bob kind=CHAT time=7
2026/10/25 17:21:09 component=Server event=broadcast_chat client=Bob time=8 chars=8 delivered=3
2026/10/25 17:21:09 component=Server event=message_delivered client=Bob kind=CHAT time=8
2026/10/25 17:21:09 component=Server event=message_delivered client=Alice kind=CHAT time=8
2026/10/25 17:21:09 component=Server event=message_delivered client=Carol kind=CHAT time=8
2026/10/25 17:21:10 component=Server event=broadcast_chat client=Alice time=9 chars=15
delivered=3
2026/10/25 17:21:10 component=Server event=message_delivered client=Alice kind=CHAT time=9
2026/10/25 17:21:10 component=Server event=message_delivered client=Bob kind=CHAT time=9
2026/10/25 17:21:10 component=Server event=message_delivered client=Carol kind=CHAT time=9
2026/10/25 17:21:10 component=Server event=broadcast_chat client=Carol time=10 chars=13
delivered=3
2026/10/25 17:21:10 component=Server event=message_delivered client=Carol kind=CHAT time=10
2026/10/25 17:21:10 component=Server event=message_delivered client=Alice kind=CHAT time=10
2026/10/25 17:21:10 component=Server event=message_delivered client=Bob kind=CHAT time=10
2026/10/25 17:21:10 component=Server event=client_disconnected client=Bob cause=leave
2026/10/25 17:21:10 component=Server event=broadcast_leave client=Bob time=11 delivered=2
2026/10/25 17:21:10 component=Server event=message_delivered client=Carol kind=LEAVE time=11
2026/10/25 17:21:10 component=Server event=message_delivered client=Alice kind=LEAVE time=11
2026/10/25 17:21:11 component=Server event=client_disconnected client=Alice cause=leave
2026/10/25 17:21:11 component=Server event=broadcast_leave client=Alice time=12 delivered=1
2026/10/25 17:21:11 component=Server event=message_delivered client=Carol kind=LEAVE time=12
```

```
2026/10/25 17:21:11 component=Server event=broadcast_chat client=Carol time=13 chars=3
delivered=1
2026/10/25 17:21:11 component=Server event=message_delivered client=Carol kind=CHAT time=13
2026/10/25 17:21:12 component=Server event=client_disconnected client=Carol cause=leave
2026/10/25 17:21:12 component=Server event=broadcast_leave client=Carol time=14 delivered=0
2026/10/25 17:21:12 component=Server event=server_shutdown reason=signal signal=interrupt
2026/10/25 17:21:12 component=Server event=server_shutdown_complete
```

A.2 Client log (Alice) — `logs/client-Alice.log`

```
2026/10/25 17:21:07 component=Client event=client_startup client=Alice server=127.0.0.1:50051
2026/10/25 17:21:07 component=Client event=client_joined client=Alice
2026/10/25 17:21:07 [t=1] Participant Alice joined Chit Chat at logical time 1
2026/10/25 17:21:07 component=Client event=message_received client=Alice kind=JOIN time=1
2026/10/25 17:21:07 [t=2] Alice: hello everyone
2026/10/25 17:21:07 component=Client event=message_received client=Alice kind=CHAT time=2
2026/10/25 17:21:08 [t=3] Participant Bob joined Chit Chat at logical time 3
2026/10/25 17:21:08 component=Client event=message_received client=Alice kind=JOIN time=3
2026/10/25 17:21:08 [t=4] Bob: hi alice, hi carol
2026/10/25 17:21:08 component=Client event=message_received client=Alice kind=CHAT time=4
2026/10/25 17:21:08 [t=5] Alice: this is alice
2026/10/25 17:21:08 component=Client event=message_received client=Alice kind=CHAT time=5
2026/10/25 17:21:08 [t=6] Participant Carol joined Chit Chat at logical time 6
2026/10/25 17:21:08 component=Client event=message_received client=Alice kind=JOIN time=6
2026/10/25 17:21:08 [t=7] Carol: ciao
2026/10/25 17:21:08 component=Client event=message_received client=Alice kind=CHAT time=7
2026/10/25 17:21:09 [t=8] Bob: bob here
2026/10/25 17:21:09 component=Client event=message_received client=Alice kind=CHAT time=8
2026/10/25 17:21:10 [t=9] Alice: leaving for now
2026/10/25 17:21:10 component=Client event=message_received client=Alice kind=CHAT time=9
2026/10/25 17:21:10 [t=10] Carol: carol says hi
2026/10/25 17:21:10 component=Client event=message_received client=Alice kind=CHAT time=10
2026/10/25 17:21:10 [t=11] Participant Bob left Chit Chat at logical time 11
2026/10/25 17:21:10 component=Client event=message_received client=Alice kind=LEAVE time=11
2026/10/25 17:21:11 component=Client event=client_shutdown client=Alice
```

A.3 Client display (Alice) — `logs/client-Alice.out`

```
You are "Alice" in Chit Chat (server 127.0.0.1:50051). Type a message, or 'leave'.
```

A.4 Client logs (Bob & Carol) — `logs/client-Bob.log`, `logs/client-Carol.log`

```
2026/10/25 17:21:08 component=Client event=client_startup client=Bob server=127.0.0.1:50051
2026/10/25 17:21:08 component=Client event=client_joined client=Bob
2026/10/25 17:21:08 [t=3] Participant Bob joined Chit Chat at logical time 3
2026/10/25 17:21:08 component=Client event=message_received client=Bob kind=JOIN time=3
2026/10/25 17:21:08 [t=4] Bob: hi alice, hi carol
2026/10/25 17:21:08 component=Client event=message_received client=Bob kind=CHAT time=4
2026/10/25 17:21:08 [t=5] Alice: this is alice
2026/10/25 17:21:08 component=Client event=message_received client=Bob kind=CHAT time=5
2026/10/25 17:21:08 [t=6] Participant Carol joined Chit Chat at logical time 6
2026/10/25 17:21:08 component=Client event=message_received client=Bob kind=JOIN time=6
2026/10/25 17:21:08 [t=7] Carol: ciao
2026/10/25 17:21:08 component=Client event=message_received client=Bob kind=CHAT time=7
2026/10/25 17:21:09 [t=8] Bob: bob here
2026/10/25 17:21:09 component=Client event=message_received client=Bob kind=CHAT time=8
2026/10/25 17:21:10 [t=9] Alice: leaving for now
2026/10/25 17:21:10 component=Client event=message_received client=Bob kind=CHAT time=9
2026/10/25 17:21:10 [t=10] Carol: carol says hi
```

```
2026/10/25 17:21:10 component=Client event=message_received client=Bob kind=CHAT time=10
2026/10/25 17:21:10 component=Client event=client_shutdown client=Bob
-----
2026/10/25 17:21:08 component=Client event=client_startup client=Carol server=127.0.0.1:50051
2026/10/25 17:21:08 component=Client event=client_joined client=Carol
2026/10/25 17:21:08 [t=6] Participant Carol joined Chit Chat at logical time 6
2026/10/25 17:21:08 component=Client event=message_received client=Carol kind=JOIN time=6
2026/10/25 17:21:08 [t=7] Carol: ciao
2026/10/25 17:21:08 component=Client event=message_received client=Carol kind=CHAT time=7
2026/10/25 17:21:09 [t=8] Bob: bob here
2026/10/25 17:21:09 component=Client event=message_received client=Carol kind=CHAT time=8
2026/10/25 17:21:10 [t=9] Alice: leaving for now
2026/10/25 17:21:10 component=Client event=message_received client=Carol kind=CHAT time=9
2026/10/25 17:21:10 [t=10] Carol: carol says hi
2026/10/25 17:21:10 component=Client event=message_received client=Carol kind=CHAT time=10
2026/10/25 17:21:10 [t=11] Participant Bob left Chit Chat at logical time 11
2026/10/25 17:21:10 component=Client event=message_received client=Carol kind=LEAVE time=11
2026/10/25 17:21:11 [t=12] Participant Alice left Chit Chat at logical time 12
2026/10/25 17:21:11 component=Client event=message_received client=Carol kind=LEAVE time=12
2026/10/25 17:21:11 [t=13] Carol: bye
2026/10/25 17:21:11 component=Client event=message_received client=Carol kind=CHAT time=13
2026/10/25 17:21:12 component=Client event=client_shutdown client=Carol
```